

LAB 01

1 Objective

This lab connects early-course topics: authenticated data, network propagation, fault-tolerant consensus, and deterministic state execution. Design and implement a minimal Layer 1 blockchain that achieves *reliable finality* under a simulated unreliable network. Every correct node must converge on the same sequence of finalized blocks, regardless of message delay, duplication, or temporary loss.

2 System overview

Each node maintains (i) a local ledger of proposed and finalized blocks, (ii) a local view of the shared application state, and (iii) links to other nodes with random delay and limited throughput. The system is driven by signed messages. Nodes communicate only through the simulated network; there is no shared memory or central authority. A valid run must show that once a block is finalized, all honest nodes permanently agree on its height and hash, and no conflicting block can ever be finalized.

3 Fault model and assumptions

The validator set is fixed for the duration of one test run. Let n be the number of validators. The system assumes $n = 3f + 1$, where at most f validators may be Byzantine.

A Byzantine validator may remain silent, send duplicate messages, send conflicting votes, propose invalid blocks, or replay previously observed messages. Byzantine validators cannot forge signatures of correct validators and cannot find collisions for the selected hash function.

The network is *partially synchronous*. Before an unknown stabilization time, messages may be delayed, duplicated, reordered, or dropped. After stabilization, messages between correct nodes are delivered within a known upper bound Δ .

Safety must hold at all times: no two distinct blocks at the same height may both be finalized by correct nodes. **Liveness** is required only after the network has stabilized and a correct proposer is eventually selected.

4 Core concepts

State. The state is the current data the network agrees on. Model it as key-value records (e.g., account, balance, or a stored variable) where each value is the latest content. Each valid transaction transforms one state into the next; each block records one deterministic state transition. All nodes produce the same result when executing the same ordered list of valid transactions.

Example. Initially empty. Alice sets "Alice/message"="hello"; later Bob sets "Bob/message"="hi". Nodes that apply both, in order, end with the same two-entry state.

Transaction. A transaction is a signed request by a participant to change part of the state that they own. Nodes verify the signature before applying the change.

Example. “Alice updates “Alice/message” to a new value and signs this request.” If the signature is valid, nodes may apply it.

Block. A block is a proposed, ordered collection of transactions applied on a known parent. It references the parent hash and commits to the resulting state. Blocks link by hash to form a single history.

Example. A block contains ten verified transactions and a commitment hash that represents the post-execution state.

Vote. A vote is a signed statement by a validator supporting a specific block at a specific height. Votes occur in two phases: *Prevote* and *Precommit*. A block becomes final only with sufficient Precommits.

Example. A validator prevotes for block 42 after verifying it; later it precommits once a majority also prevotes for the same block.

Ledger and View. The ledger is the ordered record of all finalized blocks and the states they produced. Each node’s view is its current belief about which blocks are accepted or pending. Once a block is final, honest nodes’ views converge on the same ledger.

Example. After several rounds, all nodes agree block 42 is final and contains Alice and Bob’s updates; ledgers and state commitments match.

5 Cryptography layer

This layer defines identities, authentication, and commitments used across the system. It constrains what *must* hold but does not prescribe libraries or code.

Identities. Each participant and validator owns a public/secret key pair. Choose one signature scheme (e.g., Ed25519 *or* secp256k1) and use it consistently for all signed messages in a run. Validator public keys and the validator set are fixed for the duration of a test.

Message authentication. Every protocol message that affects state or consensus is signed: transactions, block headers, and votes. Use explicit *context strings* (domain separation) so a signature valid for one message type cannot be reused for another (e.g., `TX:chain_id`, `HEADER:chain_id`, `VOTE:chain_id`). Include identifying fields in what is signed (e.g., for a vote: height and block hash) to prevent replay at other heights.

Hashing and commitments. Choose a collision-resistant hash (e.g., SHA-256 or BLAKE2). Each block header commits to the resulting state via a single state hash (your chosen commitment). If you use a Merkle tree for the state, define a canonical leaf encoding and left-right order so identical maps produce identical roots.

Deterministic encoding. Define a single, unambiguous byte encoding for data included in hashes or signatures (field order, endianness, string/bytes handling). Different nodes must produce the same bytes for the same logical content.

Security assumptions. Adversaries cannot forge signatures under the chosen scheme or find hash collisions in your chosen hash. Random seeds in the simulator do not supply cryptographic randomness.

Examples (conceptual). – A transaction’s signature covers sender, target, and requested change under context `TX:chain_id`. – A header’s signature covers parent hash, height, and state commitment under context `HEADER:chain_id`. – A vote’s signature covers (height, block hash, phase)

under context `VOTE:chain_id`. A signature from one context must not validate in another.

6 Network layer

The network models unreliable communication:

- Messages may be delayed, duplicated, reordered, or dropped.
- Each node limits outbound rate and may temporarily block overactive peers.
- Headers are broadcast before bodies; a body may be sent only after the receiver accepts the matching header.
- All network events (send, drop, delay, block, unblock) are logged with timestamp, node ID, type, and height.

Your simulator supports a configurable number of nodes (≥ 8). Connections and delay patterns may be random or file-driven.

6.1 Locking and vote rules Each correct validator must obey the following rules:

1. It signs at most one `PREVOTE` for each `(height, round)`.
2. It signs at most one `PRECOMMIT` for each `(height, round)`.
3. It accepts and prevotes only for a block with a valid parent, valid transactions, correct state hash, correct proposer, and valid signature.
4. When a validator observes a quorum of `PREVOTE` messages for a non-NIL block, it locks that block and records the corresponding round.
5. A validator locked on block *B* may prevote a different block only after it observes a quorum of valid `PREVOTE` messages for that different block in a later round.
6. A validator must not sign a vote from an incorrect chain, height, round, phase, or domain.
7. Nodes must ignore duplicate votes and must count at most one valid vote from each validator for a given `(height, round, phase)`.

7 Execution layer

Apply transactions deterministically to the shared state. You may choose any reasonable internal representation (map/table/list) provided:

- Each transaction affects only data owned by its sender and must carry a valid signature in the `TX:chain_id` domain.
- Given the same ordered set of valid transactions, all nodes compute identical results.
- Each block header commits to the resulting state with a hash produced by your deterministic encoding.

No gas, fees, or parallel execution are required. Focus on correctness and consistent commitment.

8 Determinism and logging

Re-running the same configuration must yield byte-identical logs and the same final state hash. Logs are the primary evidence of correctness and include every consensus or network event that influences the outcome. Provide a script that runs the same scenario twice and checks equality of logs and final state.

9 Testing requirements

Include both:

- **Unit tests** for cryptographic verification (signature/context checks), state update rules, vote counting, and block validation.
- **End-to-end tests** that simulate varied network conditions and confirm:

Test ID	Scenario	Required outcome
T1	Normal run, no faults	All correct nodes finalize same chain and state hash
T2	Duplicate/reordered messages	No double-counted votes, no conflicting finalization
T3	Invalid signature / wrong domain	Message rejected, reason logged
T4	Replayed/duplicate transaction	Applied at most once
T5	Drop/delay before synchrony	Safety holds; no double finalization at same height
T6	Proposer silent/crashed	Round timeout triggers, chain continues
T7	Up to f Byzantine validators equivocate	No conflicting finalization
T8	Same seed rerun twice	Byte-identical logs and final state hash

All tests must be reproducible and callable from a single entry point.

10 Submission regulation

- Students create a folder Lab01_ID1_ID2_ID3_ID4_ID5 containing the contents following:

```
src/
tests/
logs/
config/
```

README.md
REPORT.pdf

- **README.md** explains how to build and run the simulator and how to invoke tests.
- **REPORT.pdf (max 10 pages):**
 - * Information about team members.
 - * Project structure.
 - * Introduction: summary of the system purpose and the main design choices.
 - * System Design: diagram and explanation of modules and message flow. Provide a detailed explanation of the functioning and flow of your system, highlighting key components and interactions.
 - * Implementation Notes: explanation of how each guard, gossip strategy, and consensus step was realized. Include some non-trivial bugs and how you resolved them.
 - * Testing and Results: description of all test cases, expected and observed outcomes, and discussion of anomalies.
 - * All links and books related to your submission must be mentioned.
 - * **DO NOT** insert your source code in the report.
 - * The report can be written in Vietnamese or English.
- Compress the above folder into **Lab01_ID1_ID2_ID3_ID4_ID5.zip** for submission.
- Submission with wrong regulation will result in a "0" (zero).
- Plagiarism and Cheating will result in a "0" (zero) for the entire course and will be subject to appropriate referral to the Management Board for further action.

Good luck with your lab assignment!
